# NORTHERN UNIVERSITY
## OF BUSINESS & TECHNOLOGY KHULNA

**Department of Computer Science and Engineering**

# Project Title: AI-Based Smart Quiz System.

Course Code: CSE 4204
Course Title: Mobile Computing Lab

**Submitted By:**


**Team Name** : CSE4204-8A-T03
**Section** : 8A


**GitHub Repository**: https://github.com/sudipto224/AI-based-Smart-Quiz-System

**Submitted to:**


Md. Riaz Mahmud
Designation: Assistant Professor
Department of CSE


**Date of Submission: 07/09/2026**

# TEAM INFORMATION

| | |
|---|---|
| Official Team Name | CSE4204-8A-T03 |
| Section | 8A |
| Project Title | AI-Based Smart Quiz System |
| GitHub Repository | https://github.com/sudipto224/AI-based-Smart-Quiz-System |

## Team Information:

| Role | Full Name | Student ID |
|---|---|---|
| Team Leader + Backend | Sudipto Das | 11220320830 |
| Artificial Intelligence | Rakibur Rahman Mishan | 11220320829 |
| Database + Frontend | Aditi Roy | 11220320846 |

## 1. Testing Approach

The Quizora system is being tested using a combination of manual functional testing, role-based authorization checks, API request testing, database verification, AI output evaluation, browser-based UI inspection, responsive testing through Chrome DevTools, and regression testing after bug fixes. The objective is to verify expected behavior under valid, invalid, and unexpected conditions while protecting the database and avoiding destructive security tests.

- Manual browser testing for teacher and student workflows.
- Postman or Thunder Client for HTTP request and error-response testing.
- phpMyAdmin/MySQL inspection for data persistence, relationships, updates, and deletion.
- Laravel logs for AI model/error verification without exposing API keys.
- Chrome DevTools for desktop, tablet, and mobile responsive checks.
- Retesting after fixes and regression testing of previously working modules.

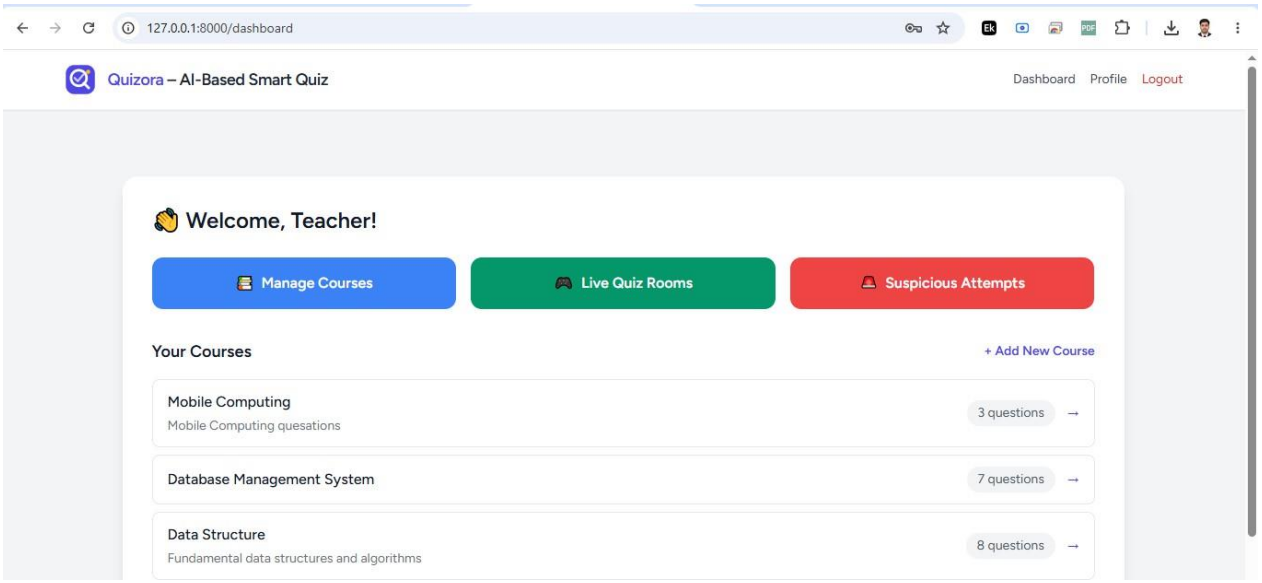



Figure: Overall Functional Test Execution of the Quizora Teacher Dashboard

## 2. Tested Modules

- Authentication and role-based access (teacher/student).
- Profile management.
- Teacher course management and question management.
- Normal timed quiz, result review, leaderboard, and suspicious-attempt monitoring.
- Course-aware AI single-question generation.
- Material-based PDF/PPTX quiz generation.
- AI Distractor Quality Checker.

- Teacher-created Live Quiz Rooms with join code, waiting room, synchronized start, submission, result, and participant monitoring.
- Database persistence for users, courses, questions, quiz attempts, live sessions, and participants.
- UI/UX, responsive behavior, and basic security controls.

### 3. Functional Testing Summary

Core role-based navigation and the newly integrated Live Quiz Room workflow have been exercised in the working application. Teacher room creation, generated room code, student joining from a separate student account, waiting-room polling, participant count updates, room opening, and quiz start routing were verified during development. Existing course, question, timed quiz, result, leaderboard, and AI modules remain part of the Week 10 regression set.

Verified development outcomes include:

- Teacher and student accounts can access their respective dashboards.
- Teacher access to the student live-quiz route is blocked with HTTP 403, confirming role separation.
- A teacher live room was created successfully and generated the code DS-3974.
- A student joined the room from an incognito student session and reached the waiting room.
- The teacher room view updated the joined count from 0 to 1.
- After the teacher started the quiz, the student was redirected from the waiting room to the live quiz route.
- AI Distractor Quality Checker correctly differentiated a deliberately poor MCQ from a strong MCQ.

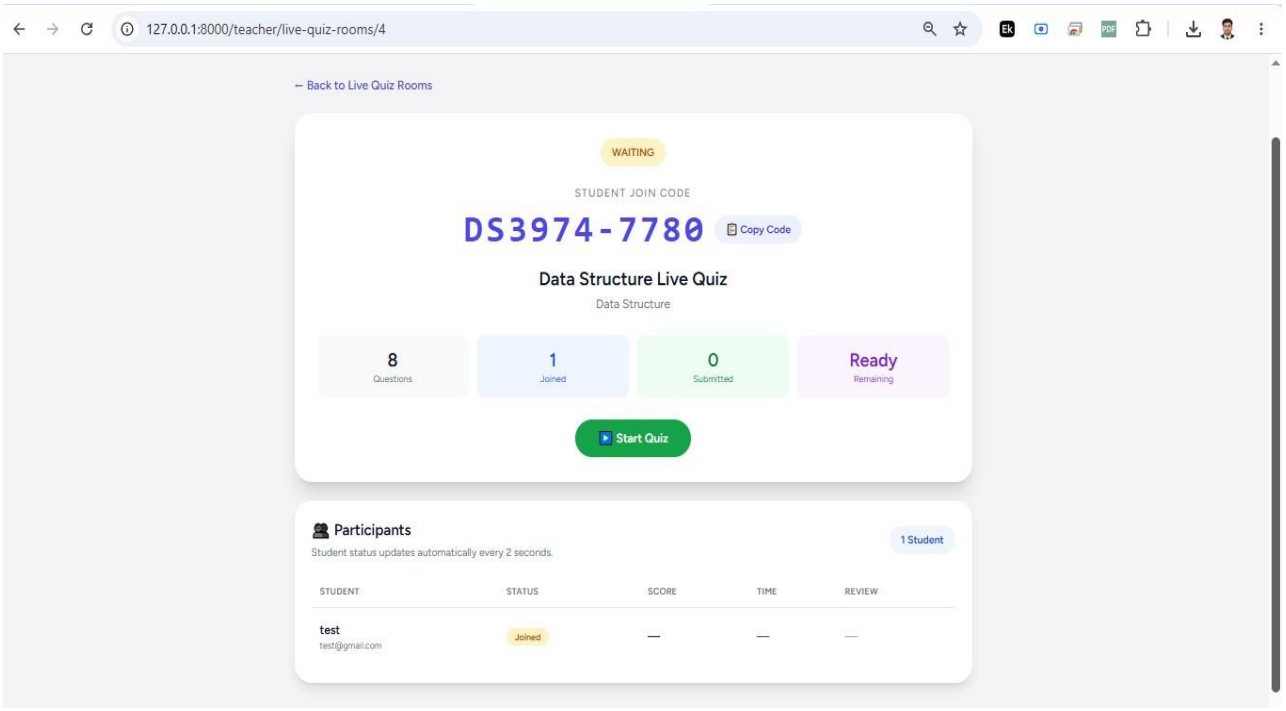



Figure. Teacher live quiz room showing successful room creation and a joined participant.

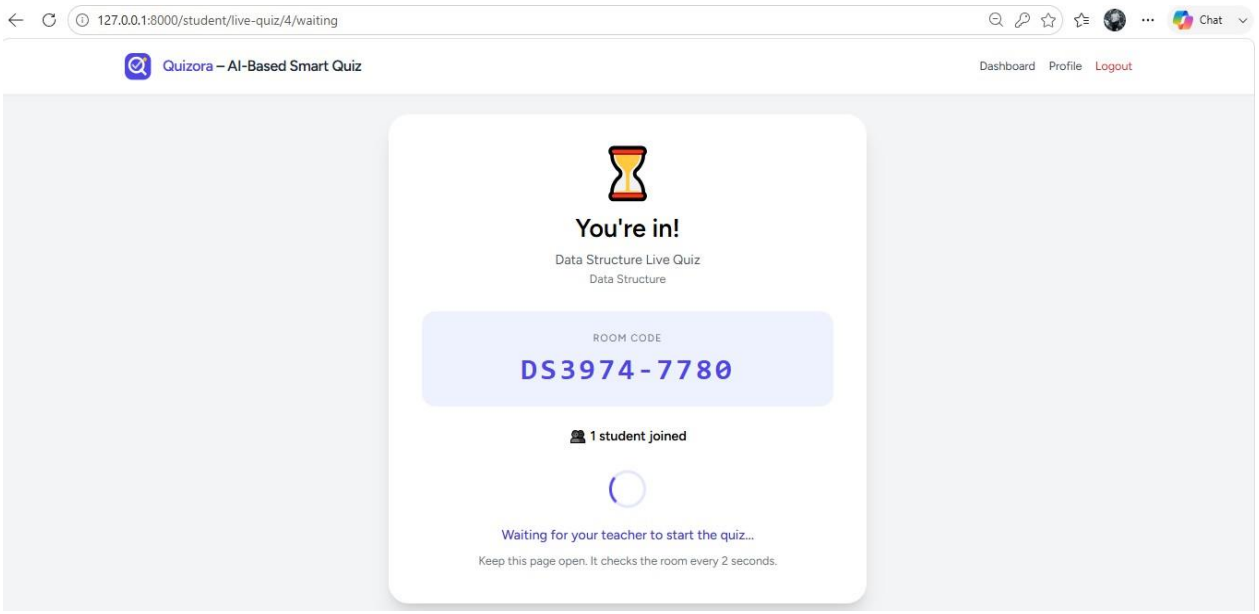



Figure. Student waiting room showing successful live quiz joining and the working student participation flow.

## 4. Authentication and Authorization Testing

Authentication testing covers valid login, invalid credentials, empty fields, logout, protected route behavior, and role restrictions. Authorization is especially important because Quizora separates

teacher and student functionality through the is_teacher role flag and controller/middleware checks.

A verified authorization test showed that a teacher account attempting to open the student live-quiz join page received HTTP 403, while the same page loaded successfully after signing in with a student account. Additional negative login, logout, and direct protected-route tests are included in the Test Cases document and should be re-executed before final submission.

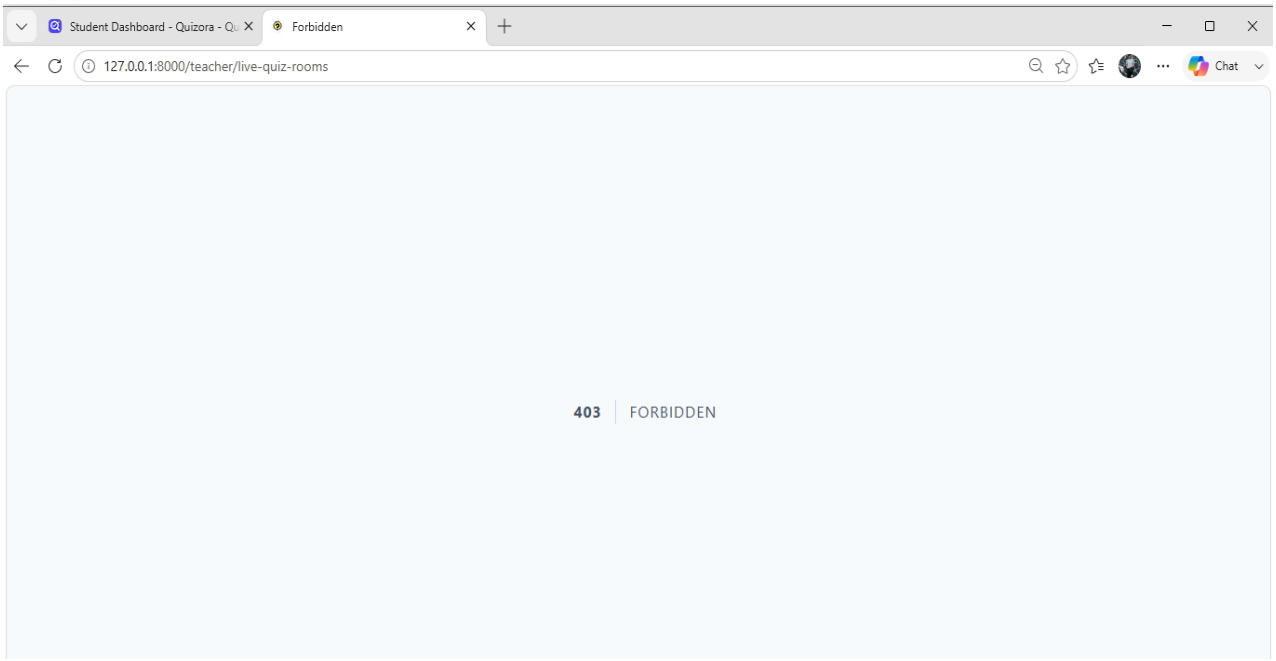



Figure. Authorization test showing a 403 Forbidden response for unauthorized student access to a teacher-only route.

## 5. API Testing

API-level testing should be performed with Postman or Thunder Client on representative GET, POST, PUT/PATCH, and DELETE operations. The tests should verify HTTP status codes, validation messages, authentication/session requirements, missing parameters, invalid parameters, and AI request error handling. Because the application uses Laravel session authentication and Inertia for many browser flows, API testing should focus on routes that can be safely exercised without destructive data loss.

Recommended API checks include course/question operations, AI quality-check request validation, live-room status endpoints, and unauthenticated requests. Record both successful and failed responses in the Test Cases PDF.

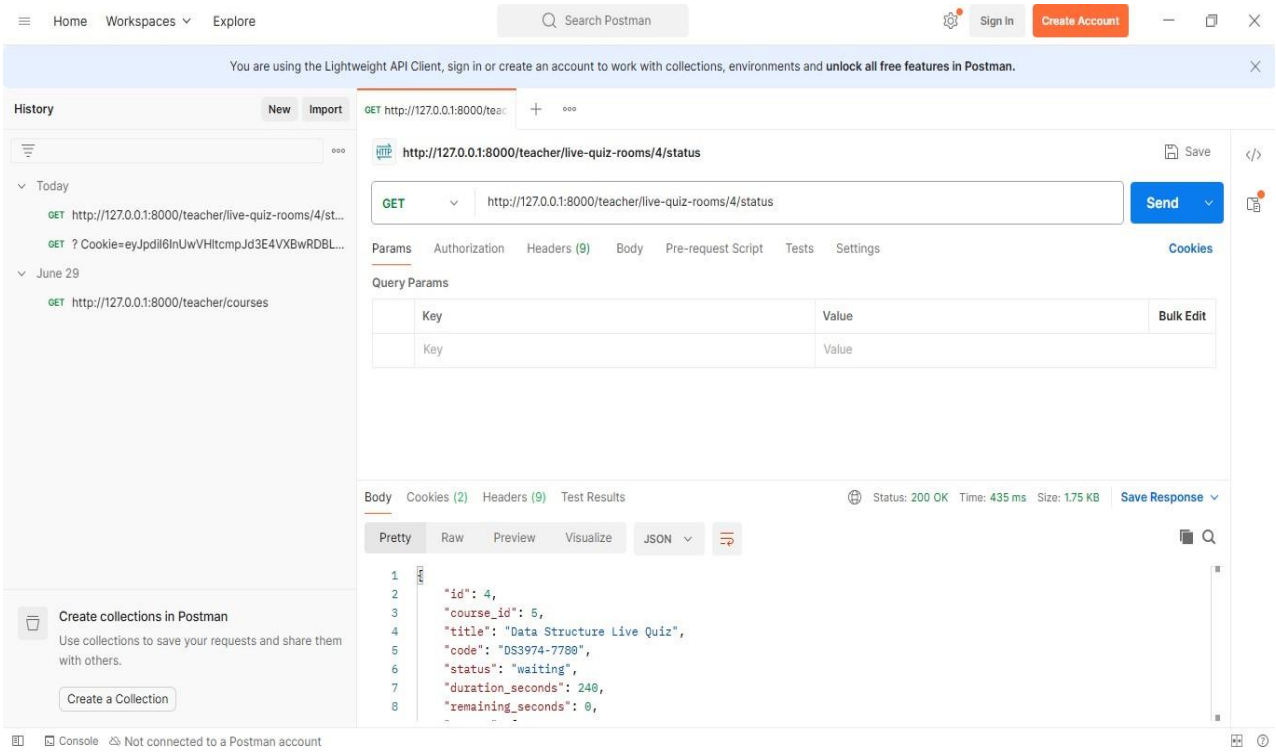



Figure. Successful GET API request in Postman showing HTTP 200 OK status and live quiz room JSON response.

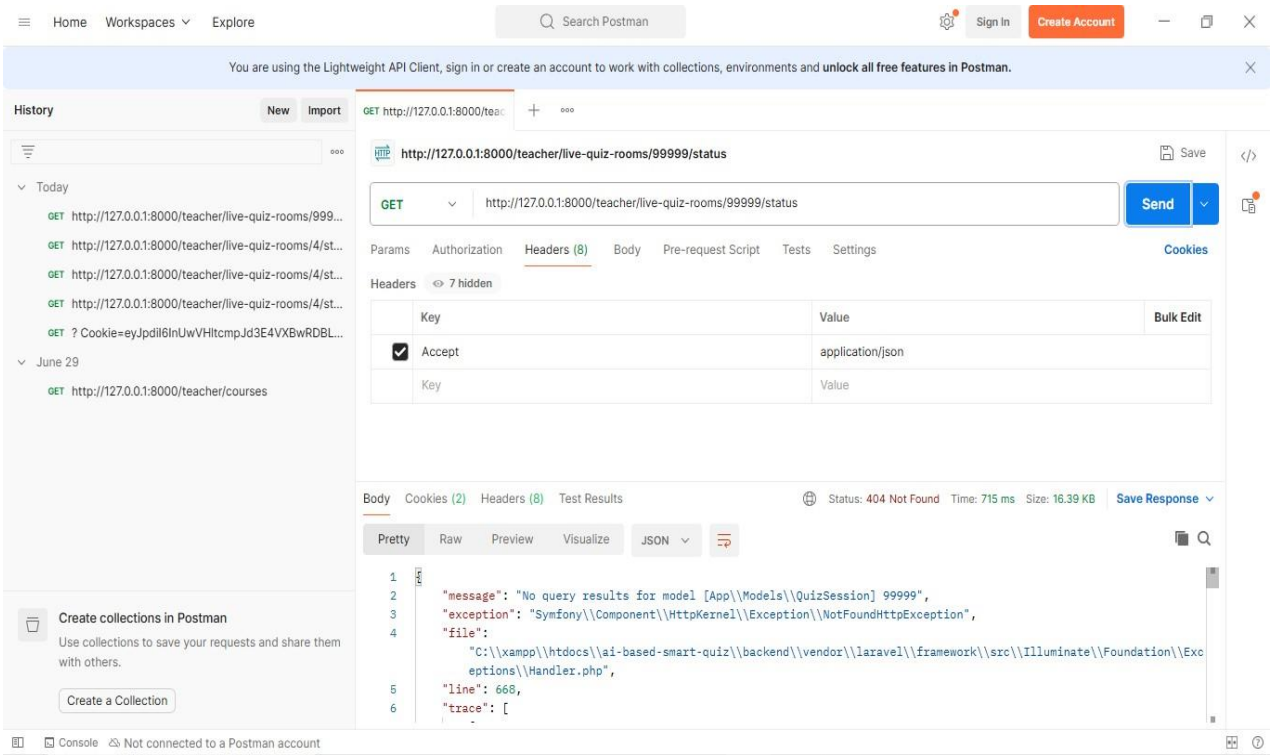



Figure. Invalid API request in Postman showing HTTP 404 Not Found response.

## 6. Database Testing

Database testing verifies that application actions are correctly reflected in MySQL. The current project persists users, courses, questions, quiz attempts, selected answers, live quiz sessions, room-question mappings, and live quiz participants. The Live Quiz testing already demonstrated successful persistence because the teacher room and student participant were available across separate sessions and participant counts updated from the stored data.

- Verify course create/update/delete values in the courses table.
- Verify MCQ options, correct answer, and explanation in the questions table.
- Verify quiz score, answers JSON, timing, suspicious flag, and live-session relationship in quiz_attempts.
- Verify live room data in quiz_sessions and participant data in quiz_session_participants.
- Verify foreign-key relationships and ownership restrictions.
- Verify invalid or duplicate inputs are rejected by validation/constraints where applicable.

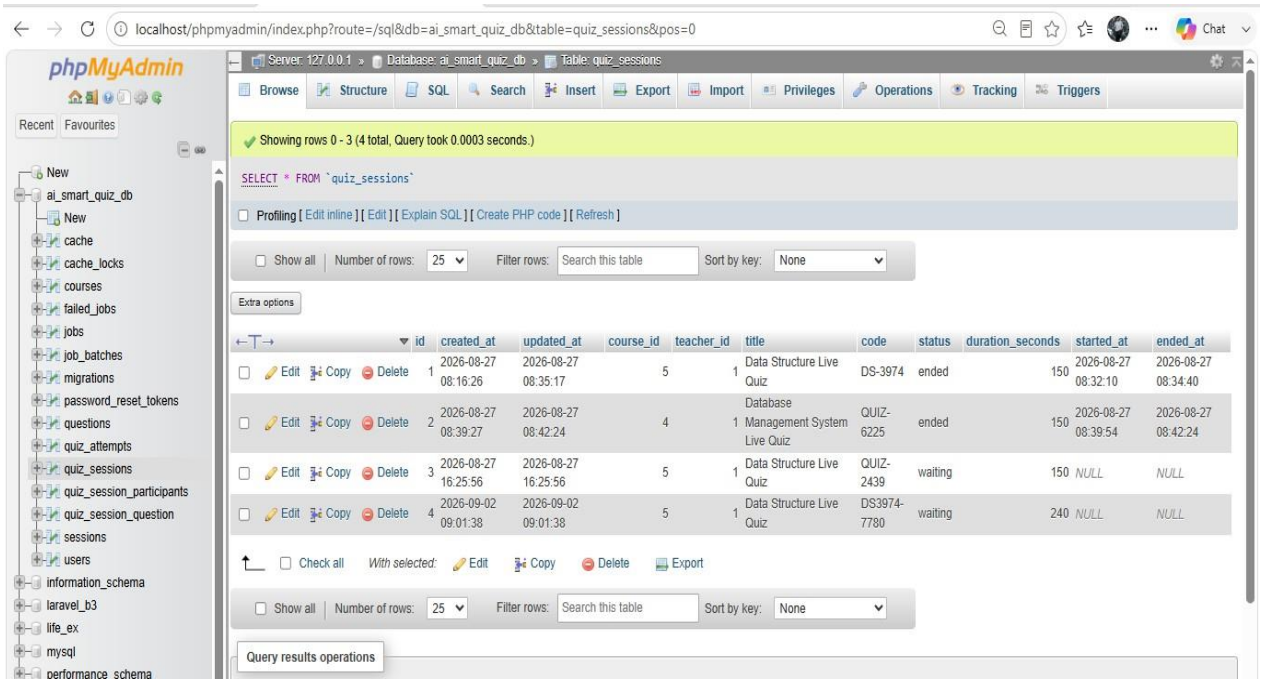



Figure. phpMyAdmin evidence showing stored live quiz session data in the MySQL database.

## 7. AI Testing Summary

Quizora currently uses Groq Cloud with the openai/gpt-oss-120b model for AI-assisted quiz functionality. AI testing covers the course-aware single-question generator, material-based question generation, and the AI Distractor Quality Checker. The evaluation criteria are relevance,

accuracy, consistency, response quality, response time, handling of unexpected input, and error handling when the AI service is unavailable or returns an unexpected response.

Two quality-checker cases have already been verified:

| Input Type | Observed Result | Interpretation | Status |
|---|---|---|---|
| Deliberately poor MCQ with irrelevant distractors | 25/100 - Poor | The checker identified irrelevant choices, duplicate meaning/style imbalance, and an easily identifiable correct answer. | Verified |
| Strong OOP MCQ with plausible distractors | 90/100 - Good | The checker recognized clear wording, one defensible answer, relevant distractors, and balanced options. | Verified |

At least ten distinct AI inputs are included in the Test Cases document. Cases not yet re-executed for Week 10 are intentionally marked Pending rather than being reported as completed.

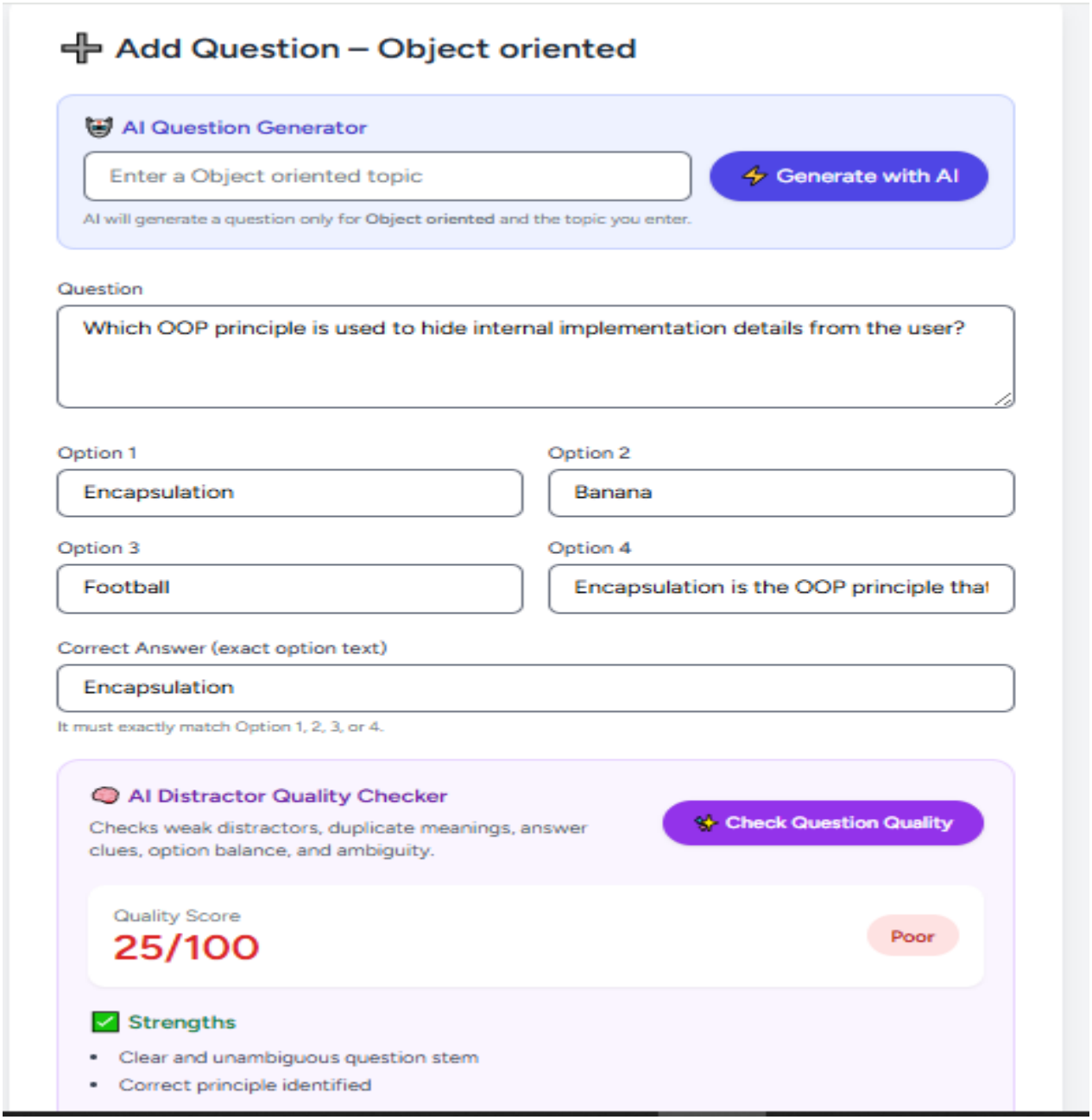



Figure. AI Distractor Quality Checker identifying a poor-quality MCQ with a score of 25/100.

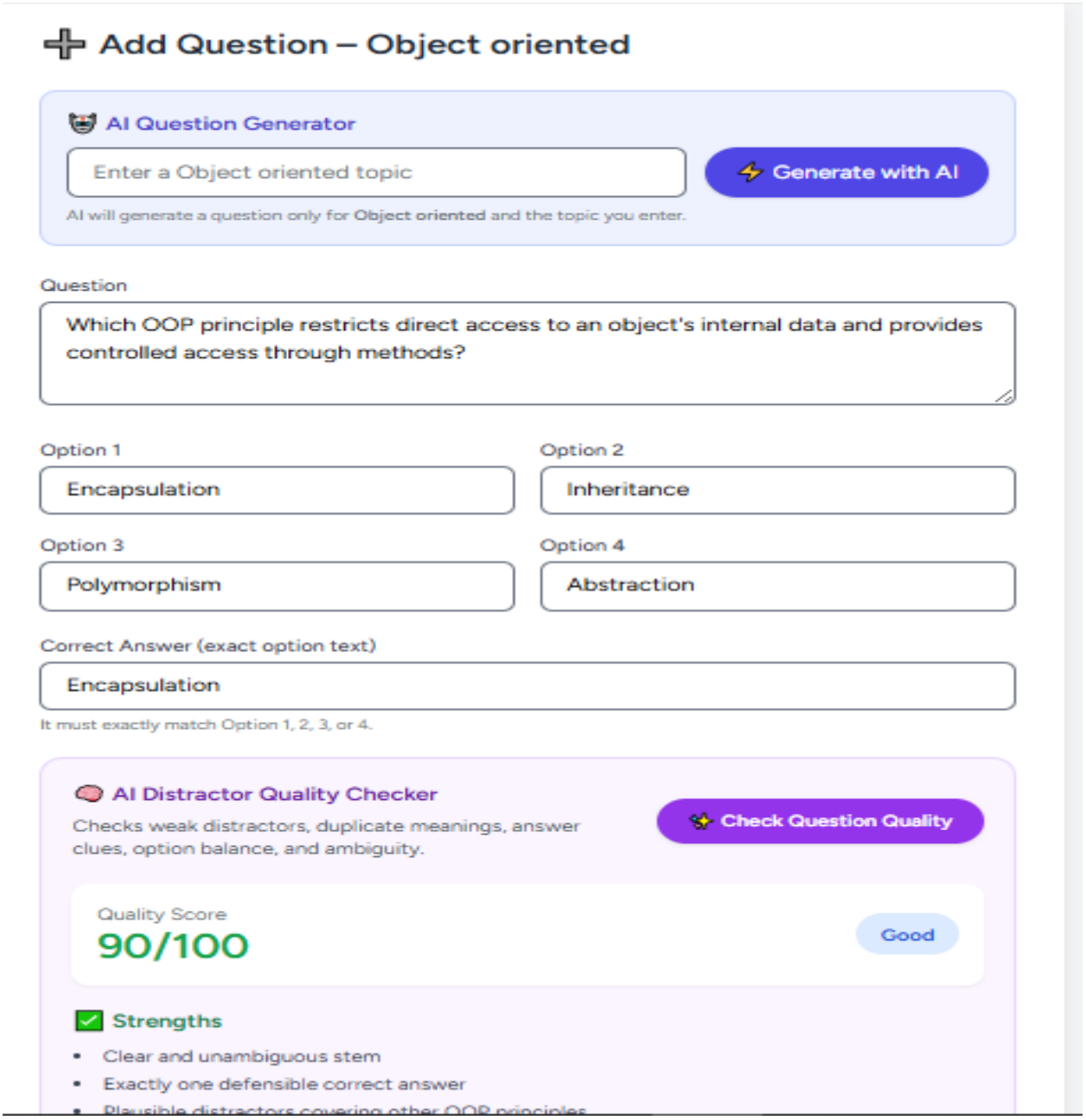



Figure. AI Distractor Quality Checker identifying a strong-quality MCQ with a high quality score.

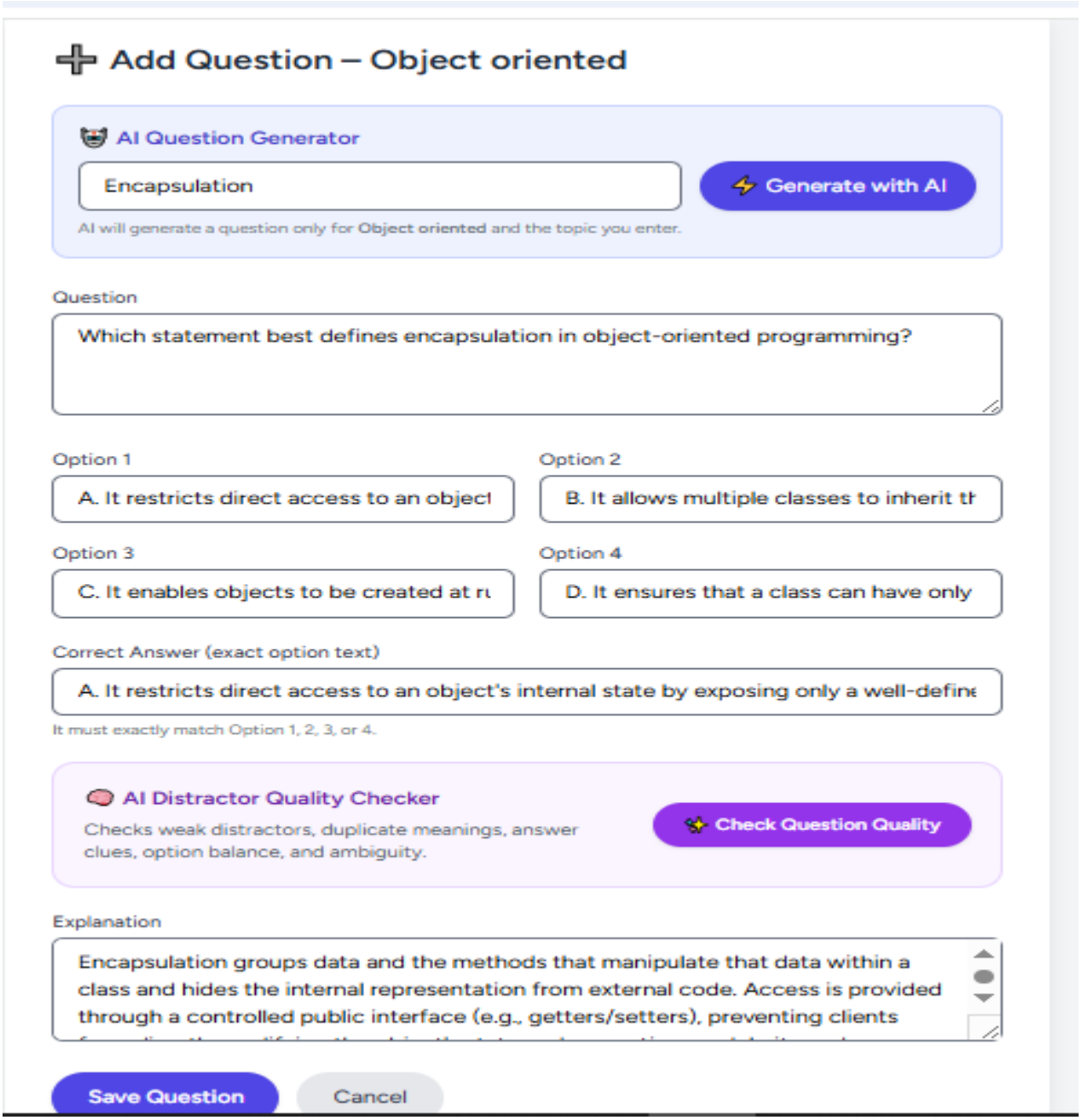



Figure. Course-aware AI question generation result for the Object Oriented course using the topic "Encapsulation."

### 8. UI/UX Testing

UI/UX testing covers navigation, buttons, forms, error messages, loading states, empty states, page layout, readability, and consistency across teacher and student interfaces. During live-room integration, blank pages caused by missing or misplaced React page components were identified and fixed. Dashboard navigation was also improved by adding direct buttons for Live Quiz Rooms on the teacher dashboard and Join Live Quiz on the student dashboard.

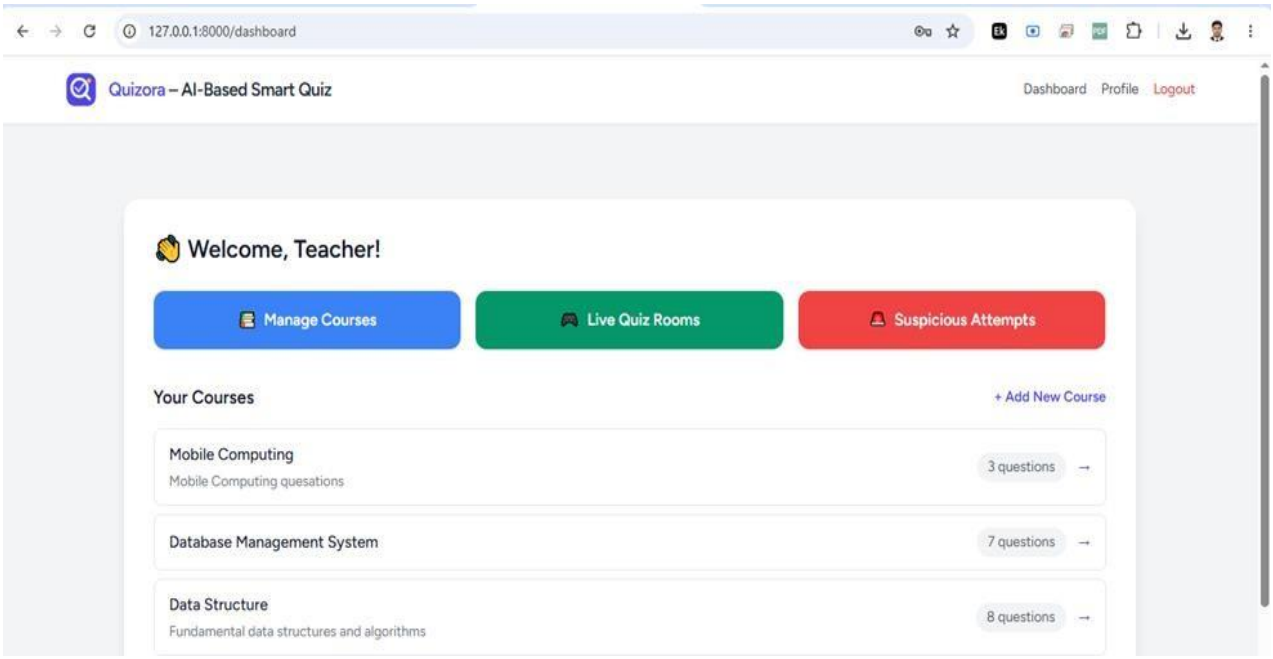



Figure. Teacher dashboard interface showing the direct navigation button for the Live Quiz Rooms feature.

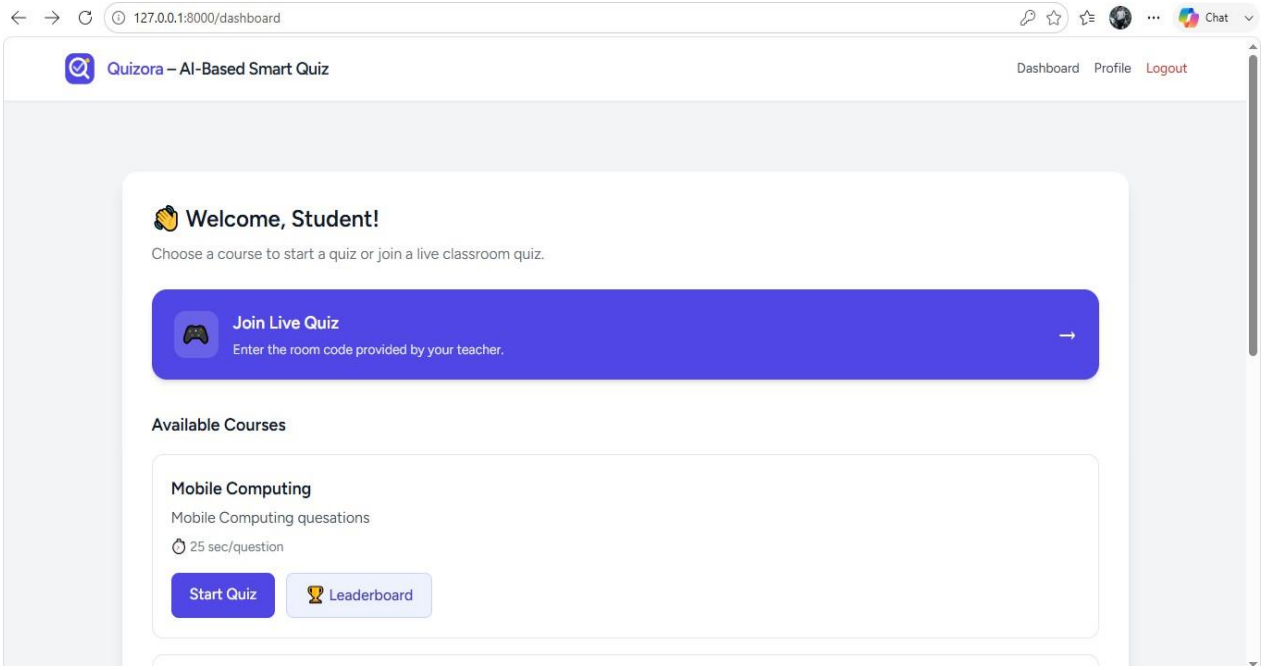



Figure. Student Dashboard showing direct Join Live Quiz navigation.

## 9. Responsive Testing Summary

Responsive testing must be completed on at least desktop, tablet, and mobile widths. Priority pages are the dashboards, forms, live quiz room, live quiz question page, AI interfaces, course cards, tables, and result pages. Chrome DevTools Device Toolbar can be used to reproduce

common screen sizes. Any wrapping, horizontal overflow, clipped buttons, unreadable text, or unusable tables should be recorded as a bug and retested after correction.

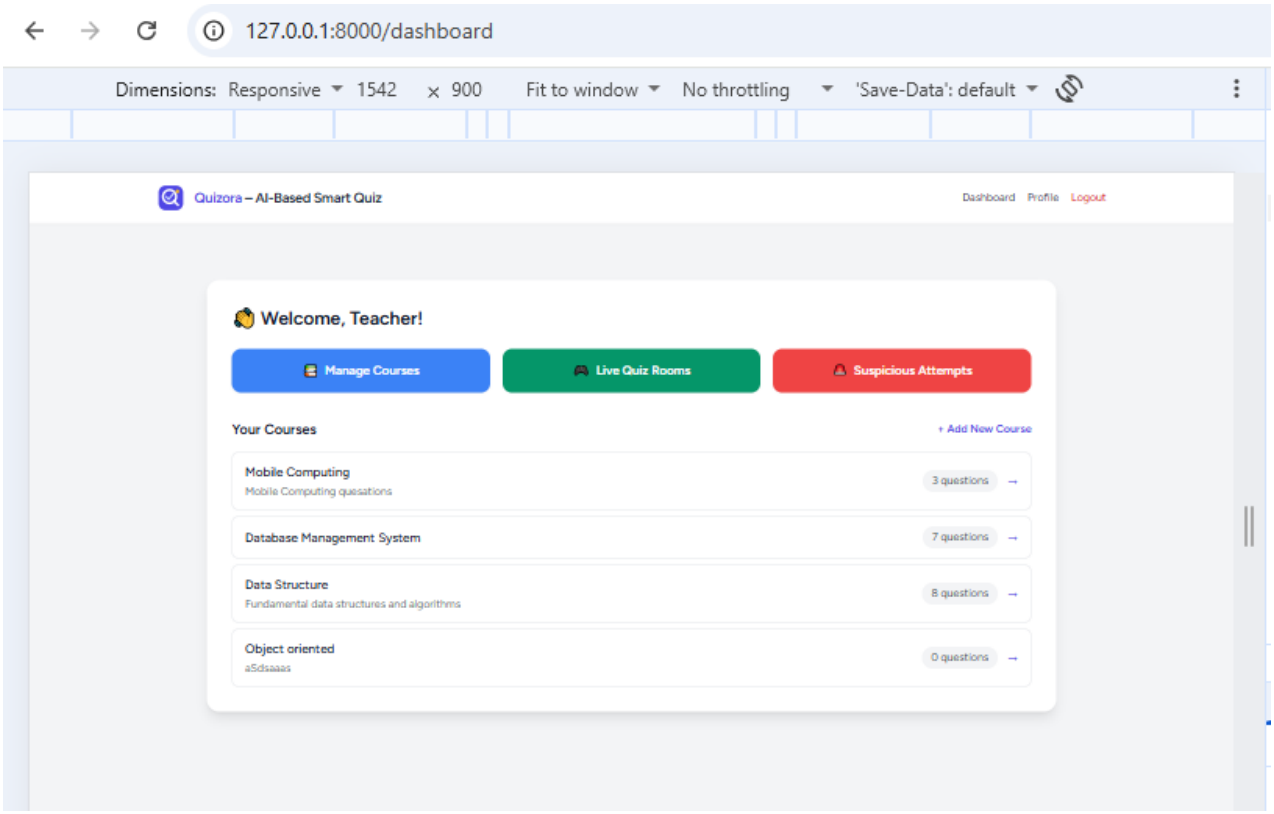



Figure. Desktop responsive test of the Quizora Teacher Dashboard at 1542 × 900 resolution.

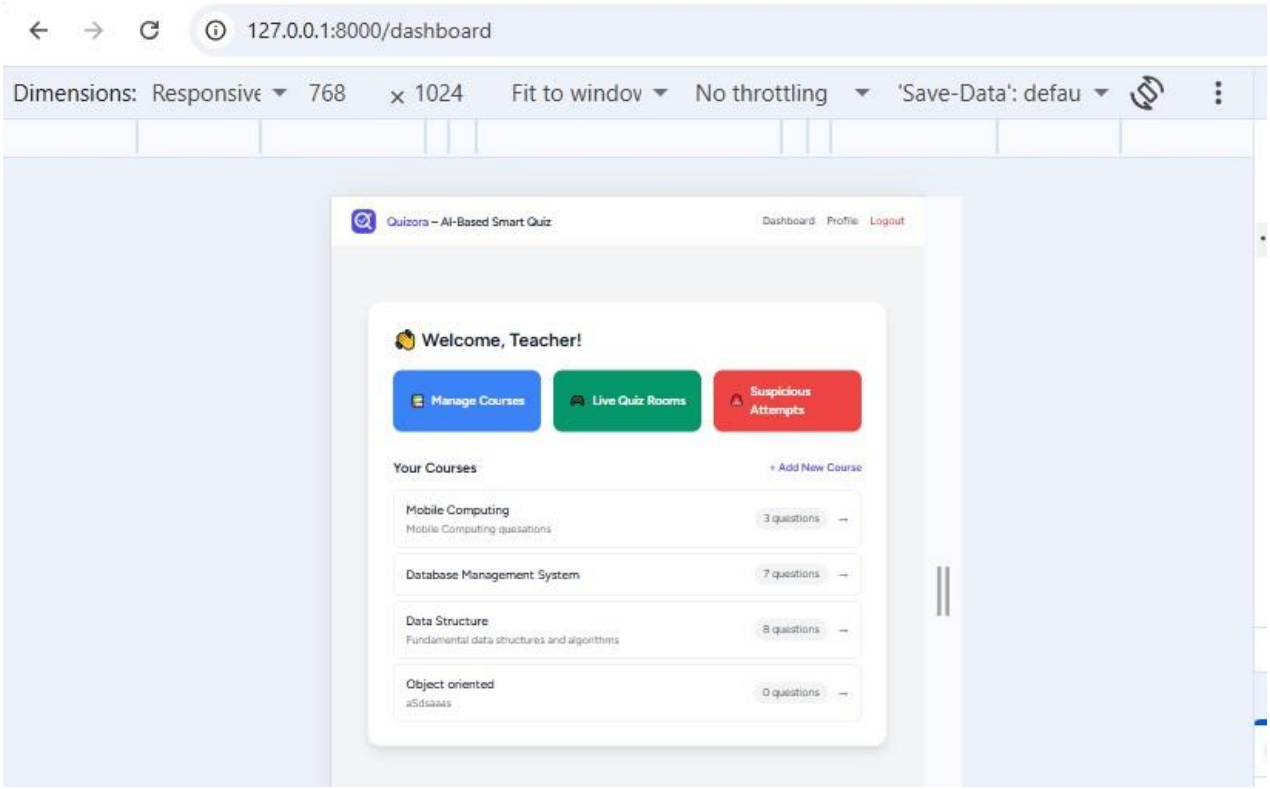



Figure. Tablet responsive test of the Quizora teacher dashboard at 768 × 1024 resolution.

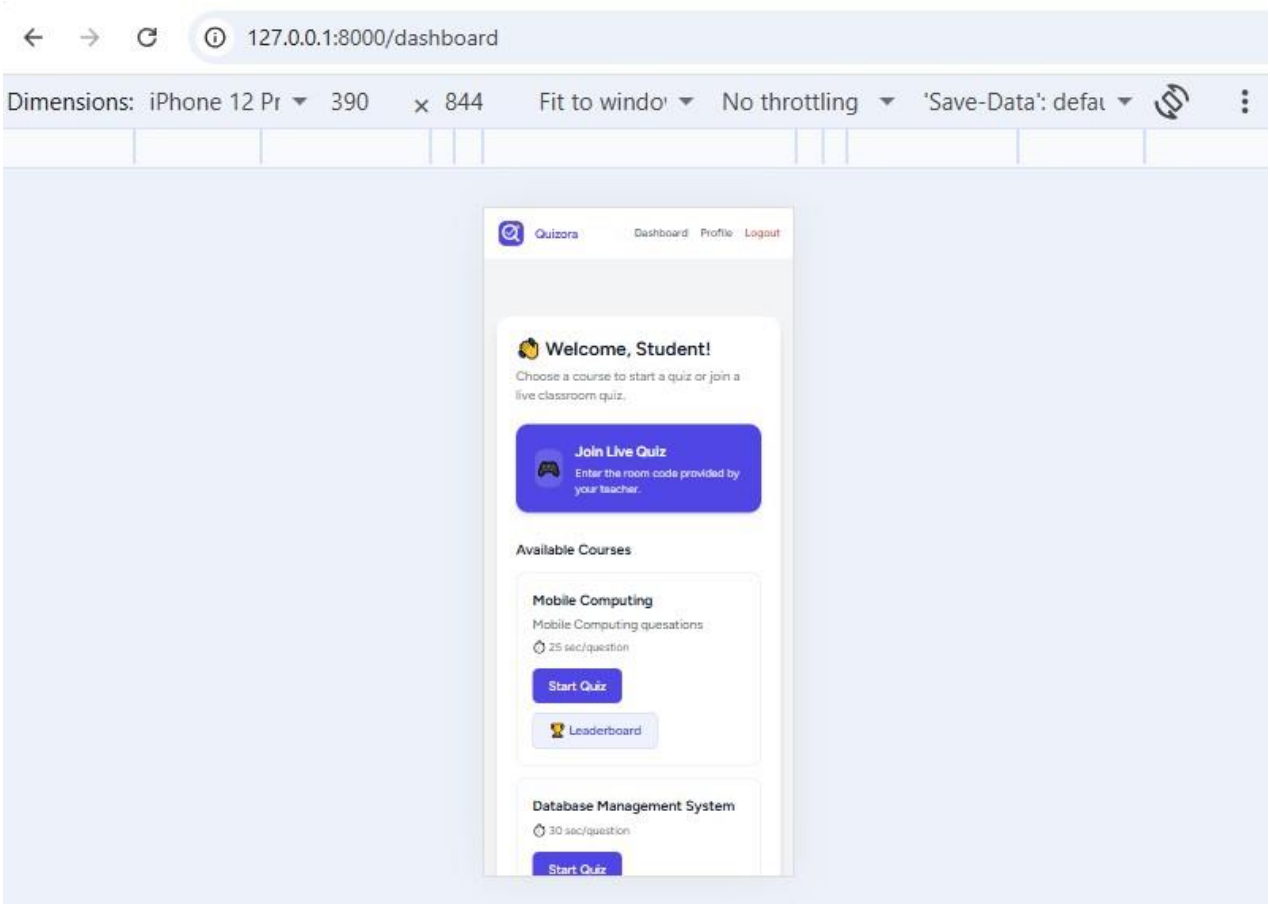



Figure. Mobile responsive test of the Quizora Student Dashboard at iPhone 12 Pro (390 × 844) resolution.

## 10. Security Checks

The Week 10 security review is limited to safe, non-destructive checks. Passwords should remain hashed by Laravel authentication, protected pages should require authentication, role-restricted routes should reject unauthorized users, API keys should remain server-side, and sensitive values should stay in environment configuration rather than source code. The Groq API key must not appear in frontend JavaScript, screenshots, reports, or GitHub commits.

- Verify password values are hashed in the users table.
- Verify .env is not committed to GitHub.
- Verify Groq API key is read from environment/config and is not returned to the browser.
- Verify student/teacher role restrictions and ownership checks.
- Verify unauthenticated access to protected routes is denied or redirected.
- Review recent GitHub commits for accidental secrets before final submission.

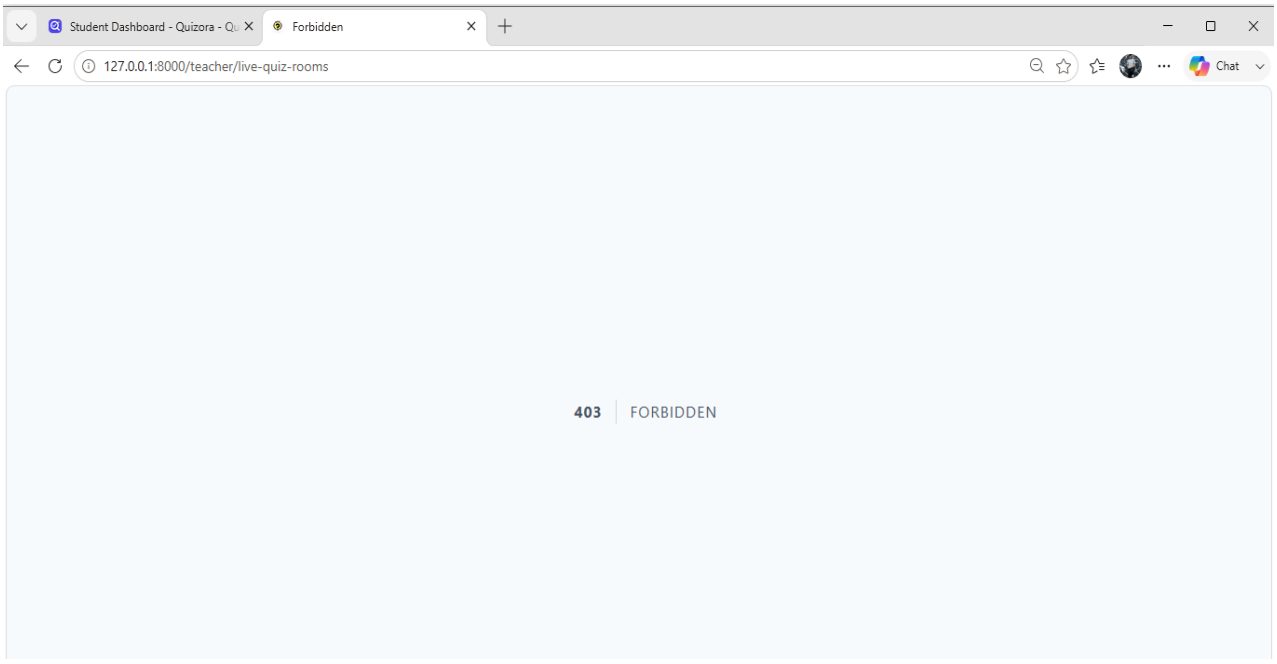



Figure. Security testing showing protected route denial for unauthorized user access (403 Forbidden).

## 11. Bug Fixes and Retesting Summary

Several real integration defects were identified during development and Live Quiz testing. These are documented in detail in the Bug Report. Important fixes included repairing an existing live-quiz database schema, adding the missing teacher room React page, moving the student live quiz component to the exact Inertia page path, persisting selected answers for result review, and correcting AI course context/model configuration. Each resolved issue should be retested and the before/after evidence retained.

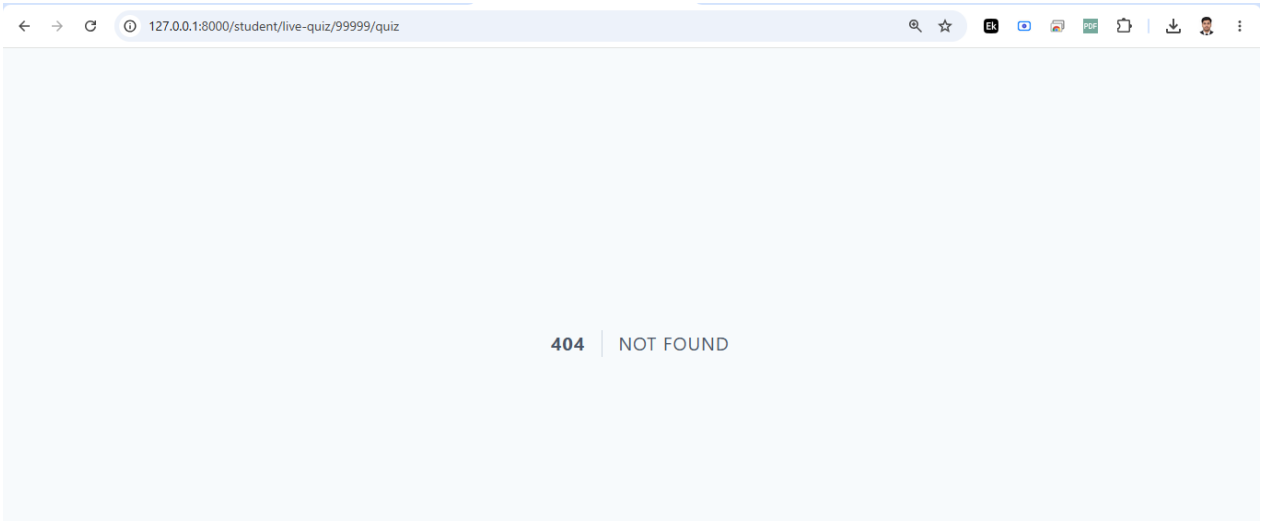



Figure: Invalid live quiz session request showing 404 Not Found response during testing.

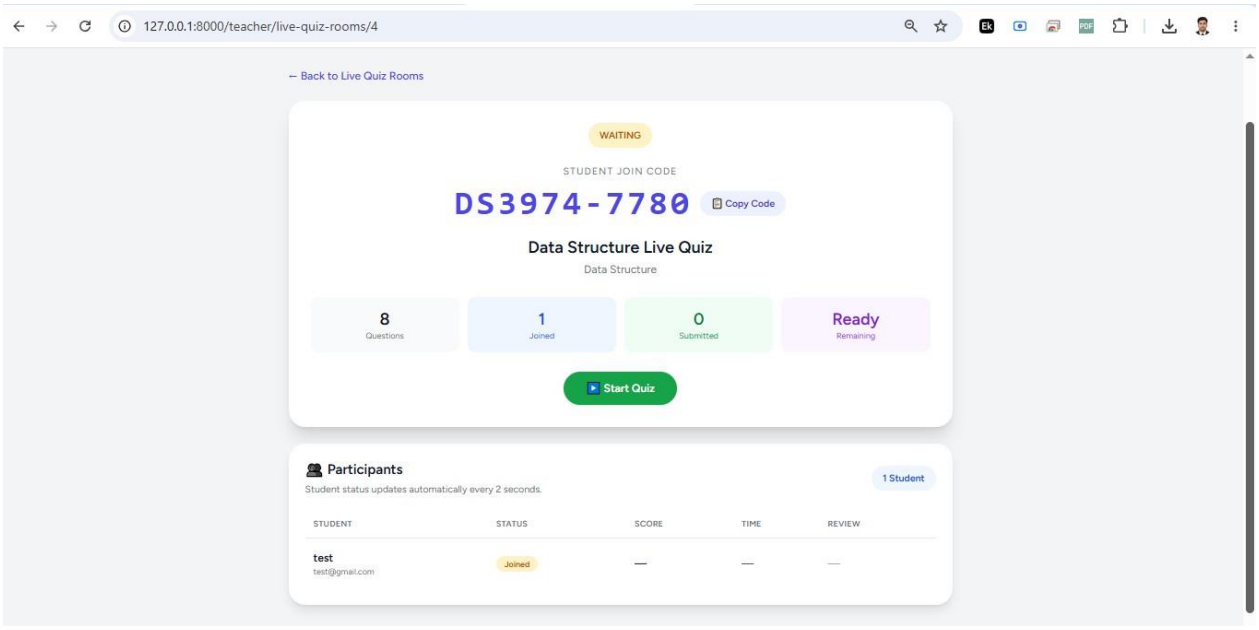



Figure. Live quiz room after bug fix, showing successful room creation and one joined participant.

## 12. Regression Testing Summary

After the Live Quiz and AI quality-checker integrations, previously working features should be rechecked to ensure the changes did not create new defects. The regression set includes registration/login/logout, profile update, course management, question management, normal timed quiz, result review, leaderboard, suspicious-attempt monitoring, AI question generation, material generation, and dashboard navigation. Current verified development flows remain operational, while the full Week 10 regression pass is listed as Pending in the Test Cases PDF until re-executed.

## 13. Test Results Summary

| **Area** | **Current Evidence** |
|---|---|
| Authentication / role separation | Valid teacher/student access and teacher-to-student route restriction verified; additional invalid-input cases pending. |
| Live Quiz Rooms | Creation, code generation, student join, waiting |

| Area | Current Evidence |
|---|---|
| | room, participant count, teacher room view, and start redirect verified; final result/submission regression to be rechecked. |
| AI Quality Checker | Poor MCQ 30/100 and strong MCQ 92/100 verified. |
| Database | Live room and participant persistence evidenced; broader CRUD/constraint checks pending re-execution. |
| API Testing | Test cases prepared; screenshots/status must be completed with Postman/Thunder Client. |
| Responsive Testing | Test plan and screenshot locations prepared; device-width execution pending. |
| Security | Role restriction observed; repository/env/password checks pending formal Week 10 evidence. |

## 14. Remaining Known Issues / Items to Verify

- Complete all Pending negative-input and API test cases with real observed results.
- Revalidate logout/CSRF behavior during regression testing.
- Verify the student live quiz result page after a full submission and timer-expiry case.
- Complete mobile/tablet responsive checks and fix any overflow if found.
- Verify AI failure handling for timeout, invalid response, and service/rate-limit errors without exposing credentials.
- Run final GitHub secret review and push meaningful Week 10 testing/fix commits.

## 15. GitHub Repository

https://github.com/sudipto224/AI-based-Smart-Quiz-System